

Character Editor & Asset Database

Category Management

- To create a new main or sub-category, add a new entry to the Char Editor Database and set the desired name and icon.
- **isItemCategory:** Creates the category panel using a grid layout group instead of a vertical layout group.
- **multiPickItem:** Allows the player to select multiple items simultaneously from the same category panel. (Like outfits from Match3D)

General Naming & Icons

- **ScriptableObject** naming is utilized for labels in the Selector, Slider, Selectorslider, and ColorSelector. This behavior can help enforce strict naming order for a well organized asset structure.
- Setting an icon for content is optional.

Content Types & UI Elements

ColorSelector

- **Setup:** Assign an ID and an array of colors for color slots.
- **Layout:** The number of colors in the array is optional. Using exactly 12 colors will make the color slots appear square (4-12 colors are recommended).
- **Color Count:** This can be configured for additional color buttons.

1: Displays no buttons.

2, 3, 4: Displays 1, 2, or 3 buttons respectively, enabling the coloring of up to 4 different parameters for a single item.

Slider

- **Setup:** Assign an ID, alongside minimum and maximum values.
- **Behavior:** Setting the minimum value to a negative number ensures the slider fill starts from the center.

- **Display:** Min/max values cap the upper and lower limits, but the visual display will always show a value scaled from 0 to 100.

Selector

There are two distinct ways to handle IDs:

- **Single ID:** Set a single base ID. The system will use _ + a number as the naming convention for calls (e.g., facetexture_0, facetexture_1, facetexture_2). The maxNumber variable is used to set the upper limit.
- **ID Array (idList):** Manually assign IDs for each entry in the idList string array. The length of the array will automatically be used as the maximum number for selection.

SelectorSlider

This is a slider-based selector that operates on int steps.

- **Setup:** Similar to the standard Selector, you can either set a single ID with a max value, or define an idList array (where the array length dictates the max value).
- **FillSlider:** Fills the slider background based on the current selection when enabled.

Item

Currently, there are two item types which can be assigned a prefab:

- **Regular Item**
- **Hair Item (Includes transform properties)**

These ScriptableObjects are designed to be extended based on project requirements.

Instantiation & Settings

Category Panels

Assigning content within the character editor database will instantiate those elements in the selected category panel at runtime.

- **Default Assets:** For Selectors and SelectorSliders, 0 is recommended for a null or default asset.

- **Example (Single ID):** face_0 can represent the default face texture.
- **Example (ID Array):** The first element of idList can be named defaultface to load the default texture via the char loader.
- **Max Value Note:** Setting maxValu manually does not take 0 into account. For example, setting maxValu to 4 yields options 0, 1, 2, 3, 4.

Content Settings Panel

Adding content to the content settings in the database will instantiate them in a separate panel for settings, which allows for customization of items and hairstyles.

- **Examples:** Adding a ColorSelector to the beard panel, a Slider for face tattoo opacity, or a Selector for clothing prints.

Utility Button Functions

Utility functions can be added via CharEditor_CategoryController. The current available functions are:

- **Close:** Closes the current category panel.
- **Undress:** Removes all content (clothes and hairstyles) from the character.
- **Remove:** Removes the currently selected content in the active category panel.
- **Save:** Handles saving the character setup.
- **Load:** Handles loading a saved character setup.
- **Light:** Changes the color palette for the editor lighting. (Note: Additional settings can be added to the color/intensity arrays within CharEditor_SceneController).
- **Randomize:** Randomizes the parameters, or equips random item(s) from the current category panel.
- **Randomize All:** Randomizes the parameters and equips random items from all category panels.

Editor Controls

The following inputs control the editor's camera and character viewing:

- **Left Click:** Rotates the character.

- **Right Click:** Rotates the camera.
- **Scroll Wheel:** Zooms the camera in and out.
- **Middle Click + Scrolling:** Adjusts the Field of View (FOV).

Implementation Notes & Suggestions

- **Saving & Loading:** CharEditor_CharLoader serves as an example demonstrating how character saving and loading can be implemented for the editor.
- **Initialization Order:** The editor must be initialized before the character to ensure all values and selections update visually without issues.
- **Naming:** ScriptableObject naming for content is important as it's used for display instead of using a script variable for label. ID in the same manner must be unique for each content to avoid conflict.
- **Asset Loading Suggestion:** It's suggested to load assets based on naming convention for selectors and step-based sliders. i.e. selector id can be set to eyeTexture and a folder for it can be populated with textures named "eyeTexture_0, eyeTexture_1..., eyeTexture_500". For naming management, **Bulk Rename Utility** can be utilized.
- **Max Limit Suggestion:** Creating a system that automatically fetches the number of assets (i.e. textures) for set IDs can further simplify the pipeline.
- **Editor Lighting:** SceneController's lighting array can be switched to a ScriptableObject based system and also utilize volume profiles and LUTs.
- **Renaming Tip:** contentPrefab naming is obsolete. It can be safely renamed without breaking any references by replacing the name in "Character Editor Database.asset".